VERSION 2.0 PRODUCTION FIRMWARE ATMEGA32 (8MHZ)

System Architecture Specification

RTOS-Based Biometric Access Control System

AUTHOR
Eng. Ahmed Akram

TARGET HARDWARE
Microchip ATmega32 (2KB SRAM / 32KB Flash)

RTOS KERNEL
FreeRTOS (Preemptive Scheduling)

REVISION DATE
October 2026

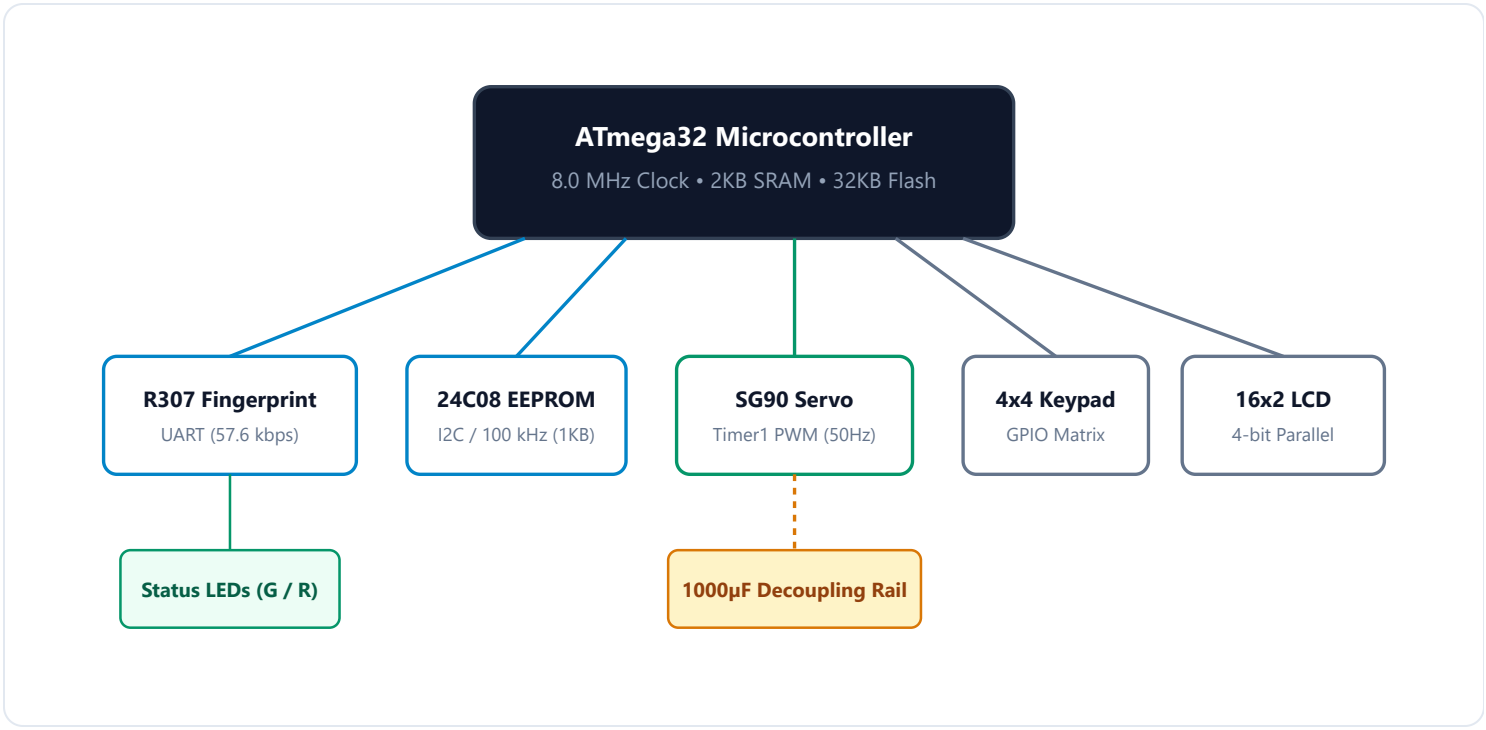
---

1. Executive Summary & Scope

This document defines the software and hardware architecture for a deterministic, real-time Biometric Access Control System designed for an 8-bit ATmega32 microcontroller. The system provides secure fingerprint acquisition, persistent role authorization, dynamic user enrollment/deletion, and physical door actuation under rigorous memory limitations (**2KB SRAM footprint**).

2. Hardware Interconnection Architecture

The ATmega32 MCU coordinates asynchronous biometric inputs, high-speed non-volatile memory queries, user interface peripherals, and inductive motor actuation over dedicated physical buses.



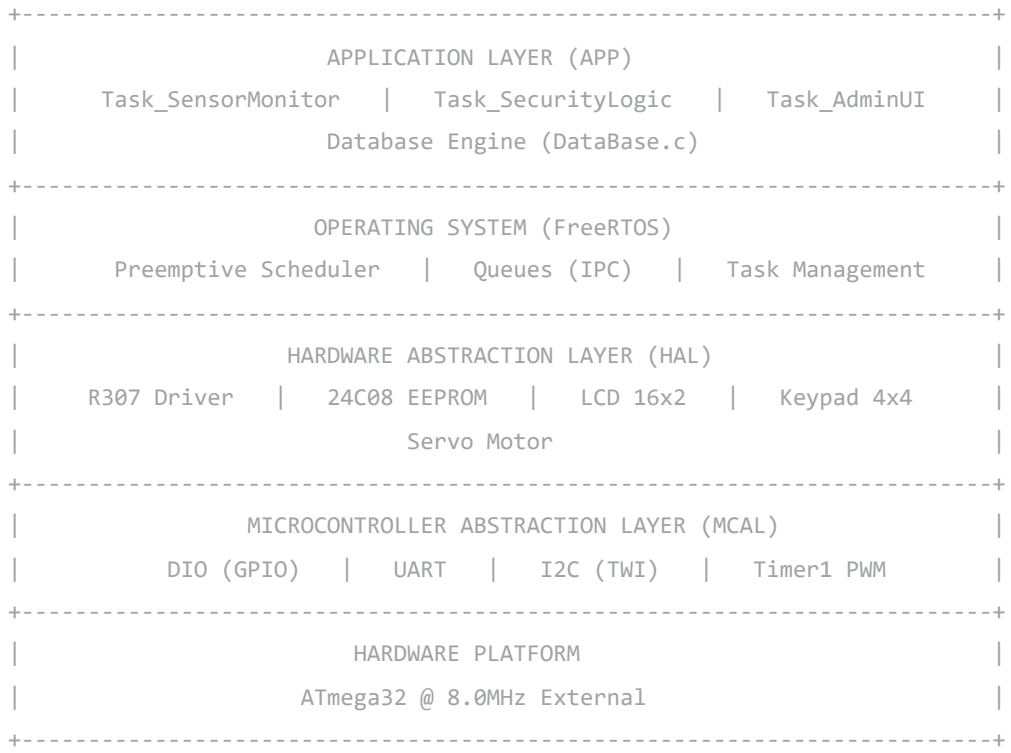
2.1 Hardware Pin Mapping

MCU PIN	PERIPHERAL DEVICE	INTERFACE BUS	FUNCTIONAL DESCRIPTION
PD0 (RXD)	R307 Fingerprint Sensor	UART (57600 bps)	Receives biometric matching packets and template acknowledgments.
PD1 (TXD)	R307 Fingerprint Sensor	UART (57600 bps)	Transmits biometric image grab and search command packets.
PC0 (SCL)	24C08 External EEPROM	I2C / TWI (100 kHz)	Synchronous clock line for non-volatile database transactions.
PC1 (SDA)	24C08 External EEPROM	I2C / TWI (100 kHz)	Bidirectional serial data line for user privilege lookups.
PD5 (OC1A)	SG90 Servo Motor	Timer1 PWM (50 Hz)	Direct physical latch actuation (1.0ms = 0° locked, 1.5ms = 90° open).
PD6	Red Indicator LED	GPIO Output	Visual alert: Access denied / unauthorized enrollment attempt.
PD7	Green Indicator LED	GPIO Output	Visual confirmation: Authorized door unlock / Admin console active.
PORTA (PA0–PA7)	16x2 Character LCD	4-bit Parallel	System telemetry and real-time administrative guidance.
PORTB (PB0–PB7)	4x4 Matrix Keypad	Matrix Scanning	User input, enrollment prompts, and dynamic deletion navigation.

> **Electrical Decoupling Engineering:**
>
> Inductive loads such as the SG90 micro-servo produce transient stall current spikes approaching 1.0A. A **1000µF electrolytic decoupling capacitor** placed across the power rail dampens inductive kickbacks and prevents MCU Brown-Out Resets (BOR) or LCD data-bus corruption without needing an isolated power supply.

3. Layered Firmware Architecture

The firmware adheres to strict architectural layering, isolating platform-dependent register manipulation from high-level security logic and UI state machines.



4. Real-Time Concurrency & Scheduling

The system executes three distinct FreeRTOS tasks governed by a fixed-priority preemptive scheduling policy. Inter-task communication follows a decoupled **Producer-Consumer Architecture**.

TASK NAME	PRIORITY	STACK DEPTH	BLOCKING MECHANISM	FUNCTIONAL DESCRIPTION
Task_SensorMonitor	3 (Highest)	160 Words	`vTaskDelay(50)`	**Producer:** Continuously polls R307 sensor and keypad 'A' key. Enqueues authenticated biometric IDs.
Task_SecurityLogic	2 (Medium)	140 Words	`xQueueReceive(portMAX_DELAY)`	**Consumer:** Suspends at 0% CPU until an ID arrives. Queries EEPROM privileges and drives servo latch.
Task_AdminUI	1 (Lowest)	160 Words	`vTaskSuspend(NULL)`	**Control Plane:** Dormant on boot. Awakened on demand by 'A' key to execute user enrollment and deletion.

4.1 Inter-Process Communication (IPC) Design

- **Handle:** `FingerprintQueue` (FreeRTOS Queue).
- **Queue Depth:** 3 Items (Type: `u16`).
- **Zero Idle Latency:** Because `Task_SecurityLogic` blocks on `portMAX_DELAY`, it yields 100% of execution time to lower tasks until an event occurs.

5. Memory Management & 2KB SRAM Optimization

Running FreeRTOS alongside multiple peripheral drivers on a 2048-byte SRAM device leaves negligible headroom for stack growth. Two critical memory optimization strategies ensure system integrity:

> **1. Static Flash Offloading (PROGMEM / PSTR):**
>
> All static UI text, debug formatting, and menu banners are pinned directly to Program Flash ROM via `PSTR()`. A custom string renderer reads bytes using `pgm_read_byte()`, freeing **350+ bytes of static SRAM** from the initialized `.data` segment.

> **2. Zero-RAM On-The-Fly Browser Algorithm:**
>
> Instead of loading the registered user table into a memory-heavy array inside the task stack (which induces immediate stack overflow), the deletion browser queries the external 24C08 EEPROM incrementally on every keypress.
>
> **Runtime Stack Footprint:**
>
> Exactly `2 Bytes` (`u16 Local_u16SelectedID`).

6. Storage Layout & Security Model

6.1 24C08 EEPROM Structure

The 24C08 EEPROM provides 8 Kbits (1024 Bytes / 1 KB) organized as 4 blocks of 256 bytes, addressed over I2C at 100 kHz.

```
typedef struct {
    u16 User_Id;       // Unique biometric ID (0xFFFF = free unallocated slot)
    u8  Privilege;     // 0 = Standard User, 1 = System Administrator
} UserRecord_t;
```

6.2 System Safeguards & Immunity Rules

- **Admin Immunity Rule:** The master administrator account (`ID: 0`) is permanently protected in firmware against accidental deletion via the Admin UI, preventing irreversible system lockout.
- **Emergency Bootstrap:** If the external EEPROM contains no registered administrator (e.g., initial factory boot), pressing 'A' routes the user directly into an administrative provisioning wizard.